

Lanzamiento de excepciones

Esteban Álvarez

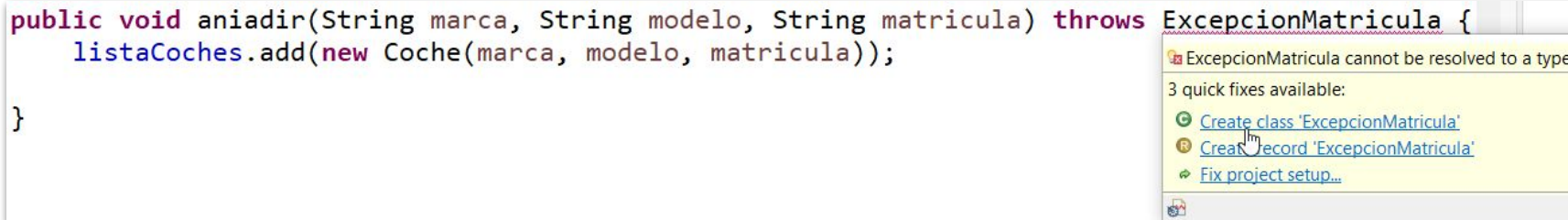
Lanzamiento de excepciones

- Hasta ahora nos limitamos a capturar **excepciones predefinidas** en Java
- Pero nuestros **métodos** también pueden **generar excepciones propias**
- Lógicamente, también se podrán **capturar** esas excepciones

Lanzamiento de excepciones

- Vamos al **método añadir** de la clase GestionCoches
- Y haremos que **lance una excepción** cuando se reciba una matrícula que no cumpla el formato 0000AAA
- Para ello añadimos en su cabecera la palabra reservada `throws` seguida del nombre de la excepción que queremos crear

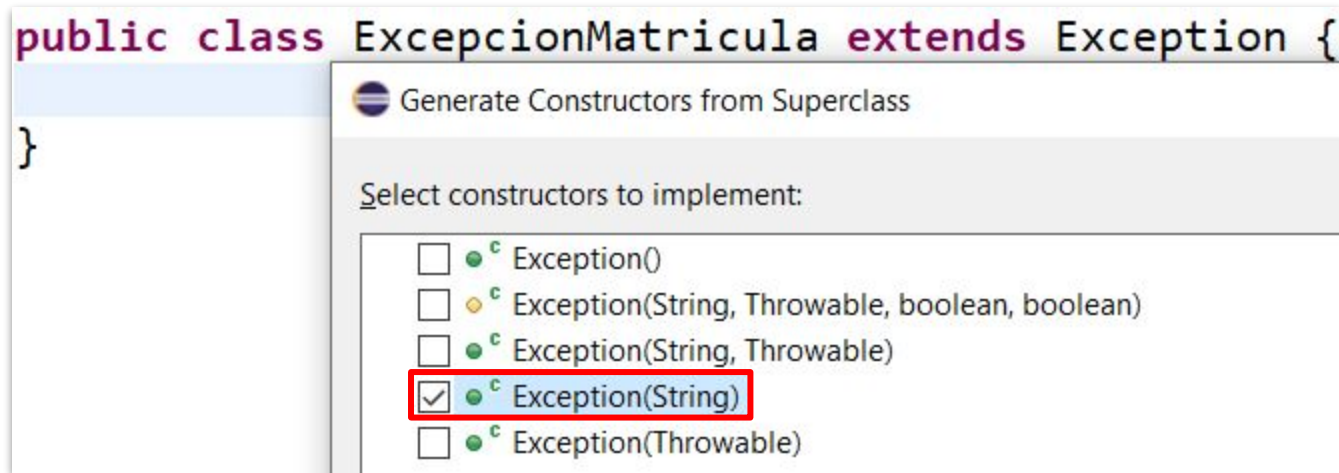
```
public void aniadir(String marca, String modelo, String matricula) throws ExcepcionMatricula {  
    listaCoches.add(new Coche(marca, modelo, matricula));  
}
```



- Como la clase no existe Eclipse muestra un error. Creamos esa clase

Lanzamiento de excepciones

- En la clase `ExcepcionMatricula`, generamos Constructors from Superclass marcando el constructor que recibe un parámetro `String`



Lanzamiento de excepciones

- Y este es el código completo de la clase `ExcepcionMatricula`

```
public class ExcepcionMatricula extends Exception {  
    public ExcepcionMatricula(String message) {  
        super(message);  
    }  
}
```

- Observa que esta excepción depende de la excepción general `Exception`

Lanzamiento de excepciones

- Volviendo al método añadir, ahora hay que decidir **cuándo lanzamos** la excepción creada
- Será cuando la matrícula no tenga el formato esperado 0000AAA
- Se usa la palabra reservada `throw`, se crea un objeto del tipo de la excepción y se pasa el String:

```
public void aniadir(String marca, String modelo, String matricula) throws ExcepcionMatricula {  
    if (matricula.matches("(\\d{4})([A-Z]{3})")) {  
        //Matrícula correcta. Se añade el coche a la lista  
        listaCoches.add(new Coche(marca, modelo, matricula));  
    } else {  
        //Matrícula incorrecta. No se añade y se lanza la excepción  
        throw new ExcepcionMatricula("Formato de matrícula incorrecto");  
    }  
}
```

Lanzamiento de excepciones

- El código comentado con **Javadoc** incluye una etiqueta sobre el lanzamiento de la excepción:

```
/**
 * Método añadir Coche Recibe los datos de un coche y lo añade a la lista si la
 * matrícula es válida
 * @param marca
 * @param modelo
 * @param matricula
 * @throws ExcepcionMatricula Si la matrícula no cumple el formato 0000AAA
 */
public void aniadir(String marca, String modelo, String matricula) throws ExcepcionMatricula {
    if (matricula.matches("(\\d{4})([A-Z]{3})")) {
        //Matrícula correcta. Se añade el coche a la lista
        listaCoches.add(new Coche(marca, modelo, matricula));
    } else {
        //Matrícula incorrecta. No se añade y se lanza la excepción
        throw new ExcepcionMatricula("Formato de matrícula incorrecto");
    }
}
```

Lanzamiento de excepciones

- Cuando otra clase (Main en este caso) quiera hacer uso del método aniadir debe tratar la excepción:

```
coches.aniadir(marca, modelo, matricula);
```

Unhandled exception type ExcepcionMatricula

2 quick fixes available:

 [Add throws declaration](#)

 [Surround with try/catch](#)

Lanzamiento de excepciones

- Cuando otra clase (Main en este caso) quiera hacer uso del método `aniadir` debe tratar la excepción
- Observa que `e.printStackTrace()` muestra como mensaje el String que se indicó cuando se lanzó la excepción en el método `aniadir`:

```
try {  
    coches.aniadir(marca, modelo, matricula);  
} catch (ExcepcionMatricula e) {  
    e.printStackTrace();  
}
```

```
ExcepcionMatricula: Formato de matrícula incorrecto  
    at GestionCoches.aniadir(GestionCoches.java:  
    at PrincipalCoches.main(PrincipalCoches.java:  
1  Añadir un nuevo coche
```

Lanzamiento de excepciones

- Como siempre, se puede modificar el mensaje de error:

```
try {  
    coches.aniadir(marca, modelo, matricula);  
} catch (ExcepcionMatricula e) {  
    System.out.println("Formato de matrícula incorrecto");  
}
```

Delegación del tratamiento de excepciones

- Otras veces los métodos pueden **no tratar las excepciones** que están obligados a tratar y **delegar el tratamiento** a quien use ese método

Delegación del tratamiento de excepciones

- El siguiente código está en la clase `GestiónFicheros` y recibe un `ArrayList` de coches para guardarlo en un fichero
- El método está obligado a tratar las excepciones relacionadas con ficheros. En concreto, cuando detecte cualquier excepción (`Exception`) devuelve `false`

```
public boolean guardarLista(ArrayList<Coche> lista) {  
    ObjectOutputStream oos=null;  
    try {  
        oos = new ObjectOutputStream(new FileOutputStream (rutaFichero));  
        oos.writeObject(lista);  
        oos.close();  
        return true;  
    } catch (Exception e) {  
        return false;  
    }  
}
```

Delegación del tratamiento de excepciones

- Pero también podría **delegar** el tratamiento de la excepción

```
public boolean guardarLista(ArrayList<Coche> lista) throws Exception{  
    ObjectOutputStream oos = null;  
    oos = new ObjectOutputStream(new FileOutputStream(rutaFichero));  
    oos.writeObject(lista);  
    oos.close();  
    return true;  
}
```

Delegación del tratamiento de excepciones

- Y quien use el método `guardarLista` debe tratar la excepción:

```
public boolean guardarCoches() {  
    GestionFicheros.guardarLista(listaCoches);  
}
```



Delegación del tratamiento de excepciones

- Y quien use el método `guardarLista` debe tratar la excepción
- De esta forma si el método `guardarLista` genera una excepción de tipo `Exception`, se trata aquí

```
public boolean guardarCoches() {  
    try {  
        GestionFicheros.guardarLista(listaCoches);  
        return true;  
    } catch (Exception e) {  
        return false;  
    }  
}
```

Excepciones

EJERCICIO

1. Partiendo del ejercicio “Sastrería” modifica el código para que los constructores de las clases Persona, Empleado y Vendedor lancen una excepción (y no se cree el objeto) si no se cumplen los criterios de validación que especifica el enunciado
 - a. Modifica la clase principal para tratar la excepción
2. Partiendo del ejercicio “Electrodomésticos” modifica el código para que el constructor de la clase Electrodoméstico lance una excepción (y no se cree el objeto) si la etiqueta energética no es correcta

Excepciones

3. Partiendo del ejercicio [Movil y SIM](#) ([Solución](#)) modifica el constructor de la clase Móvil para que lance una excepción (y no se cree el objeto) si el número de tarjetas SIM del móvil no es 1 o 2.
 - a. Modifica la clase principal para tratar la excepción
4. Modifica el método `añadirTarjeta` para que reciba el número de teléfono y la compañía. Si el número no tiene 9 dígitos o el nombre de la compañía es nulo, debe lanzar una excepción
 - a. Modifica la clase principal para tratar la excepción

FIN!